

Homework 3 - Mining Data Streams

Data Mining

AHMAD AL KHATEEB OMAR ALMASSRI
ahmadak | omaralm@kth.se

November 2025

Background

The algorithm implemented in this homework, *Reservoir Sampling Algorithm*, is inspired by L. De Stefani, A. Epasto, M. Riondato, and E. Upfal, TRIÈST: Counting Local and Global Triangles in Fully-Dynamic Streams with Fixed Memory Size, KDD'16, <https://www.kdd.org/kdd2016/papers/files/rfp0465-de-stefaniA.pdf>. The paper proposes *TRIÈST-BASE*, the insertion-only streaming algorithms for counting triangles in a graph when:

- edges arrive one-by-one,
- you cannot store all edges (graph is huge),
- you have a fixed memory budget (keep only M edges),
- you want accurate estimates of the total number of triangles.

TRIÈST-BASE is fundamentally built upon the *Reservoir Sampling* principle, which allows to only store a sample M of edges chosen uniformly at random from all edges seen so far t (instead of storing all edges), guaranteeing the property that each of the edges is stored with equal probability M/t . For detecting sampled triangles, the paper maintains an adjacency structure in which every newly added edge is examined to determine whether a new triangle is formed. Additionally, the paper derives an unbiased estimator for the true number (since only a fraction of the edges can be seen, and the number of triangles seen is smaller than the true number),

involving the scaling factor: $\xi(t) = \frac{\binom{t}{3}}{\binom{M}{3}}$ (only applies when $t > M$).

TRIÈST-BASE

Reservoir Sampling: given a stream of edges $e_1, e_2, \dots$; maintain at most M sampled edges. The first M edges are stored directly, and for each new edge e_t (where $t > M$) is included in the sample with probability $\frac{M}{t}$. If included, it replaces a uniformly chosen edge already in the sample. This ensures each of the first t edges has equal probability of being kept. **Triangle Estimation:** let S be the sample of edges, T the number of triangles found inside the sample, and t the number of edges processed so far. A triangle in the full graph appears in the sample

only if all three of its edges lie in S , which has probability $P = \frac{\binom{M}{3}}{\binom{t}{3}}$.

Program Execution

Datasets. The dataset used for the execution and result parts are fetched from <https://snap.stanford.edu/data/web-NotreDame.html>, the Stanford Large Network Dataset Collection. Download the gzip-file and locate it under `./dataset`.

How to run. First, install all the python requirements specified in `requirements.txt` 'pip install -r requirements.txt'. The program can then be run by modifying the file `run_app` (.bat on Windows or .sh on Linux) with the absolute path to the gzip-file you downloaded, and run it. This will run with the parameters specified in the batch file. If you want to try other parameters, use the CLI commands below and run the app from the file `main.py`:

```
# Run with one or multiple M-values (sample sizes) without plotting the graph.
```

```
python .\Data_mining_HWs\Homework3\main.py -d <path to the gzip file> \
-m <one or more M values - sample size> \
```

```
# Run with plot
```

```
python .\Data_mining_HWs\Homework3\main.py -d <path to the gzip file> \
-m <one or more M values - sample size> \
--plot \
```

```
# Or just type
```

```
python .\Data_mining_HWs\Homework3\main.py --help
```

Results

The dataset used for this implementation presents some statistics used to measure the estimation done by the TRIEST-BASE algorithm:

- Nodes: 325729
- Edges: 1497134
- True number of triangles: 8910005

To calculate the error of estimation, this formula was used: $Error = \frac{|estimate - true\ value|}{true\ value}$.

Reservoir sizes (sample sizes) tested were $M = [15000, 150000, 300000, 450000, 600000, 750000, 1000000]$, and the results are shown in the table below:

Sample Size M	Triangles Estimated	Relative Error
15000	64912562	6.3
150000	48361501	4.4
300000	40394632	3.5
450000	32595831	2.7
600000	26496356	2.0
750000	24150874	1.7
1000000	22954330	1.6

Table 1: Summary of observed estimated triangles for different sample sizes.

From the results, the algorithm shows high variance but still a meaningful estimate for small $M = 15k$. For large M , the reservoir contains a significant portion of the true graph, and estimates improve. Because TRIEST-BASE uses a high-variance probability correction factor

$\xi(t)$, estimates may overshoot drastically for too small M . As M increases, stability improves, and estimates approach the true value.

The plot below compares the TRIÈST-BASE estimate for each M with the true triangle count (horizontal reference line), and the relative error versus M . The trend shows decreasing relative error as M grows, as expected.

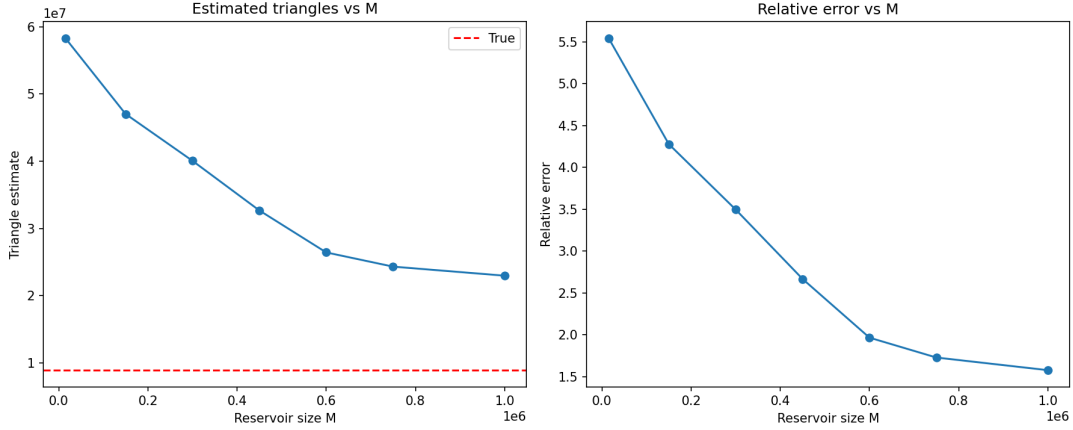


Figure 1: Estimated Triangles by TRIÈST-BASE with different M -values, and the relative error against the true value.

Questions

1. **What were the challenges you faced when implementing the algorithm?** Some of the practical challenges were the efficient edge removal where choosing a random edge with `random.choice(tuple(S))` was too slow for large M . The solution was to maintain a dedicated list plus an index map to support $O(1)$ removal. It also was a bit hard in the beginning to understand the estimating method used in the paper.
2. **Can the algorithm be easily parallelized? If yes, how? If not, why? Explain.** The algorithm cannot be directly parallelized in a straightforward way, that's because we are dependent on the number of edges processed so far for sampling new edges. Also, it would be very hard to maintain the adjacency structure between the edges.
3. **Does the algorithm work for unbounded graph streams? Explain.** Yes, because the nature of Reservoir principle of having only M edges, regardless of how large the graph becomes, allows to work on unbounded streams. The memory usage also stays constant. However, the accuracy decreases when the graph becomes sparse relative to M as the variance grows.
4. **Does the algorithm support edge deletions? If not, what modification would it need? Explain.** The algorithm doesn't support deletions. In order to do that, the algorithm must be upgraded to the TRIÈST-FD (Fully Dynamic) variant from the paper, where it includes random pairing techniques to match deletions with historical insertions, and a more complex estimator that remains unbiased even when edges disappear. So this is achievable by maintaining correctness via pairing and carefully tracking the effect of deletions on both sampling and triangle counts.